الجامعة العربية المفتوحة
Arab Open University

The Open University

# TM354: Software Engineering

## Block II: From analysis to design

**Unit 8:** The case study: part 2

# Unit Aims

- In this unit we produce a structural model, complete the analysis of the case study and produce the design for the first iteration, following an approach based on the UP.
- We show that the analysis and design is being driven from the requirements and establish some basic traceability between the requirements and design.
- We begin by producing a structural domain model in the form of a concept diagram.
- The concept diagram is a class diagram in which the classes represent concepts from the domain rather than representing software.
- This allows us to represent sets of objects that are of relevance to the problem we are trying to solve.

**Note that**: You are required to practice all SAQs and Exercises in this unit

# Section 1: Introduction

- In this unit we continue the work started in Block 1 Unit 4, where we concentrated on the two disciplines of domain modelling and requirements.
- We begin by producing a structural domain model – the conceptual model. The conceptual model is just a class diagram in which the classes represent concepts from the problem domain rather than the software.
- We will start by looking at the structure of our domain and identify which domain objects should be represented as classes, which attributes should be defined in the classes and which relationships exist between the classes.

# Section 1: Introduction

- Moving into design, we can start deciding on the assignment of responsibilities to classes and recording these decisions on the class diagram.

- This will involve the following tasks:

  - Using interaction diagrams in verifying preconditions and fulfilling postconditions of the system operations – leading to identification of finer grained operations
  - Assigning the operations to classes using GRASP patterns: Expert – responsibility assigned to a class with information to fulfil that Responsibility
  - Revisiting the class diagram to record all the decisions taken; this will be our design model.

# Section 1: Introduction

Figure 1 shows an overview of which parts of the development process we cover in this unit. In this unit we take a systematic approach in going from software requirements via a structural model to a design

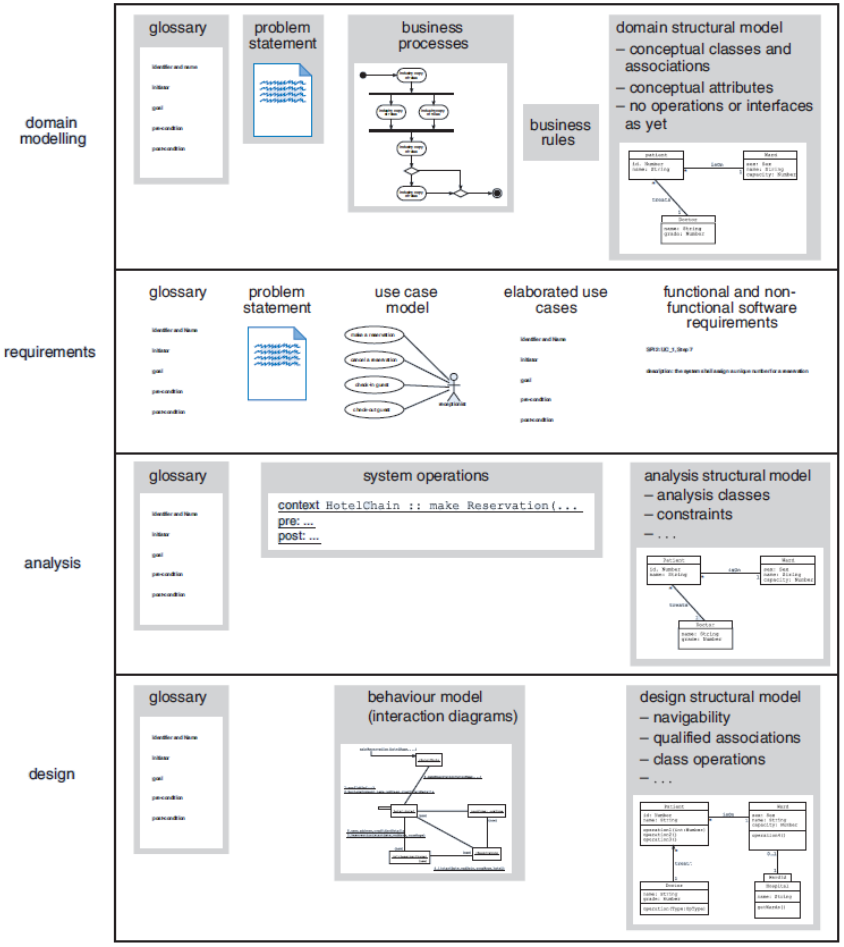


Figure 1 Parts of the development process covered in this unit

# Section 2 : Domain modelling – the conceptual model

## Identifying concepts, associations, multiplicities and attributes

- In Block 1 Unit 4 we identified the use cases to be dealt with in the first iteration of development.
- They focus on the core functionality of the business of a hotel: checking guests in and out and handling reservations.

- We will therefore start to build our conceptual model by thinking about guests, rooms and the events that determine the relationships between them.

# Section 2 : Domain modelling – the conceptual model

## Identifying concepts, associations, multiplicities and attributes

- A class diagram represents a generalization of all the possible configurations of objects in the domain.
- A conceptual model plays such a central role that it is sensible to construct it systematically, justifying each step, in order to prevent any errors creeping in.
- For this reason, we will begin with the relationships between guests and rooms, and because both of these classes have direct relationships with a hotel, we will also consider the hotel class.

# Section 2 : Domain modelling – the conceptual model

- A first attempt to produce our conceptual model can therefore borrow from Unit 5, based on the same reasoning used there to give Figure 2

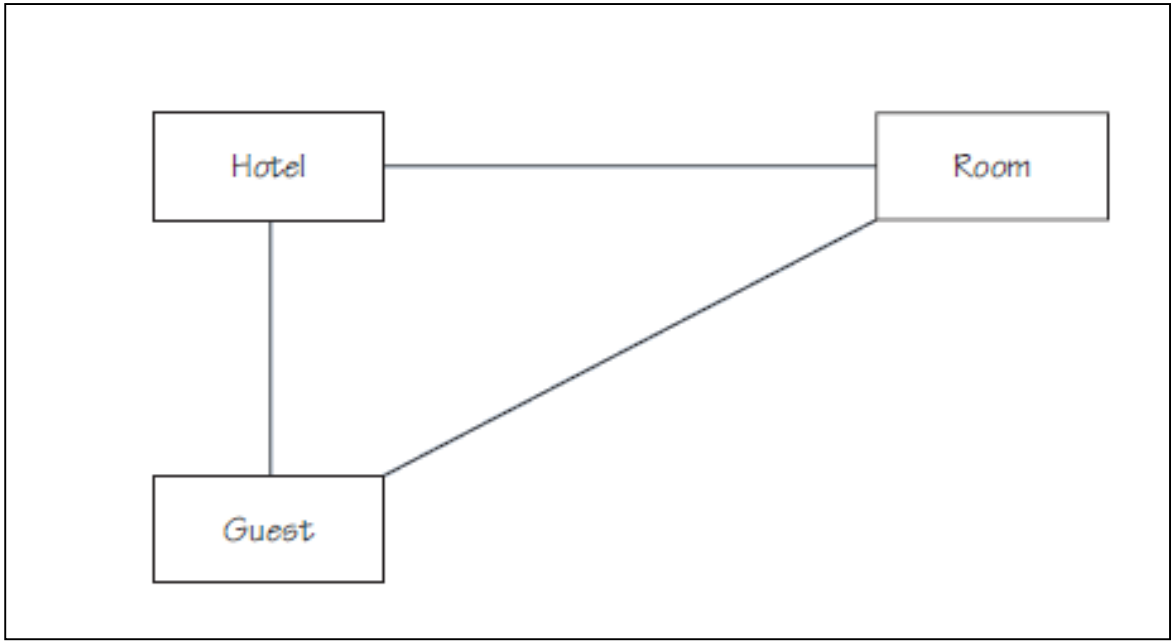


Figure 2 Some associations between Guest, Hotel and Room

# Section 2 : Domain modelling – the conceptual model

- The next step is to decide whether any other classes should be represented in the diagram, and establish how they relate to existing classes.
- We will do this by systematically working through the list of candidate classes given above.
- We will use classes to represent the main concepts in our domain.
- Tables 1 to 4 show our decisions about the candidate classes listed above.

# Section 2 : Domain modelling – the conceptual model

Table 1 Tangible objects as candidate classes

| Candidate | Represent as class? |
|---|---|
| Computer system | Not part of the domain. |
| Newspapers | No. May record as an attribute, so that we know whether the guest should have one. |
| Debit/credit card | No. We will consider it to be an attribute of *Guest* for now, but may later need to use a class. |
| Room | Already included. |

Table 2 Roles as candidate classes

| Candidate | Represent as class? |
|---|---|
| Guest | Already included. |
| Customer | No. Subsume under *Guest* (but see Exercise 1). |
| Manager | No. No information needs to be represented about managers for this iteration. Although we may need to represent managers in a later iteration, we will defer this for now to concentrate on the core classes associated with this iteration. |
| Receptionist | No. No information needs to be represented about receptionists for this iteration. |

# Section 2 : Domain modelling – the conceptual model

Table 3 Business transactions as candidate classes

| Candidate | Represent as class? |
|---|---|
| Reservation | Yes, as a reservation exists over a period of time. |
| Check in | No. Check in alters the relationship between guests and rooms, but there is no other information that needs to be recorded. |
| Check out | No. Similar argument to that for check in. |
| Cancellation | No. Causes a reservation to be deleted and perhaps a credit card to be debited, but again no other information needs to be recorded. |
| No show | No. Similar argument to that for cancellation. |
| Hotel full | No. Can be represented as an attribute. |
| Add room type/room/hotel | No. Outside scope of existing iteration. |
| Delete room type/room/hotel | No. Outside scope of existing iteration. |
| Decorating | No. Outside scope of existing iteration. |
| Building work | No. Outside scope of existing iteration. |
| Reservation request | No. Synonym for reservation. |
| Block booking | No. Outside scope of existing iteration. |
| Overflow | No. No long-term information needs to be recorded beyond that for a normal check in. |

# Section 2 : Domain modelling – the conceptual model

Table 4 Organisational units as candidate classes

| Candidate | Represent as class? |
|---|---|
| Hotel chain | Yes. |
| Hotel | Already included. |
| Nearby hotel | No. Hotels are already included. This information can sensibly be included as an association. |

- As a consequence of the decisions made in Table 3, we add Reservation to our list of classes for this iteration.
- From Table 4, we add `HotelChain`.
- Our classes are now:
  - *Guest, Room, Hotel, HotelChain, Reservation.*

# Section 2 : Domain modelling – the conceptual model

We add entries for these classes to our (currently empty) glossary, as shown in Table 5.

Table 5 Initial glossary for the hotel system

| Term | Category | Definition |
|---|---|---|
| *Guest* | Concept | A person who reserves a *Room*, stays in a *Room* and is responsible for the bill. The *Room* may be occupied also, for example by the *Guest*'s family. |
| *Room* | Concept | A physical room in a *Hotel*. |
| *Hotel* | Concept | One of the hotels in the *HotelChain*. |
| *HotelChain* | Concept | The company which manages the *Hotel*s overall. |
| *Reservation* | Concept | A record of the fact that a particular *Room* has been booked for a given period by a given *Guest*. |

# Section 2 : Domain modelling – the conceptual model

- A conceptual model corresponding to our analysis so far is shown in Figure 3.
- We choose to add RoomType as a concept rather than as an attribute because, as you will see later, it participates in a number of associations.
- Sometimes decisions like this can simply be based on insight, whereas in other cases they will be taken following further iteration.

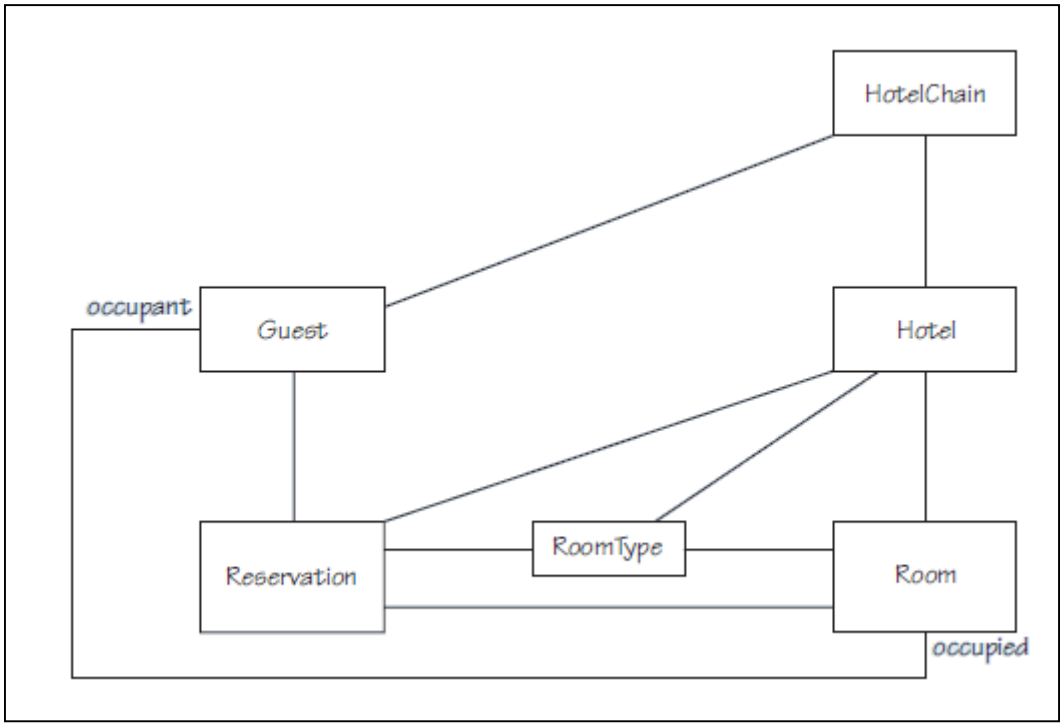


Figure 3 Conceptual model for the hotel system

# Section 2 : Domain modelling – the conceptual model

The conceptual model in Figure 4 shows the multiplicities for each of the associations

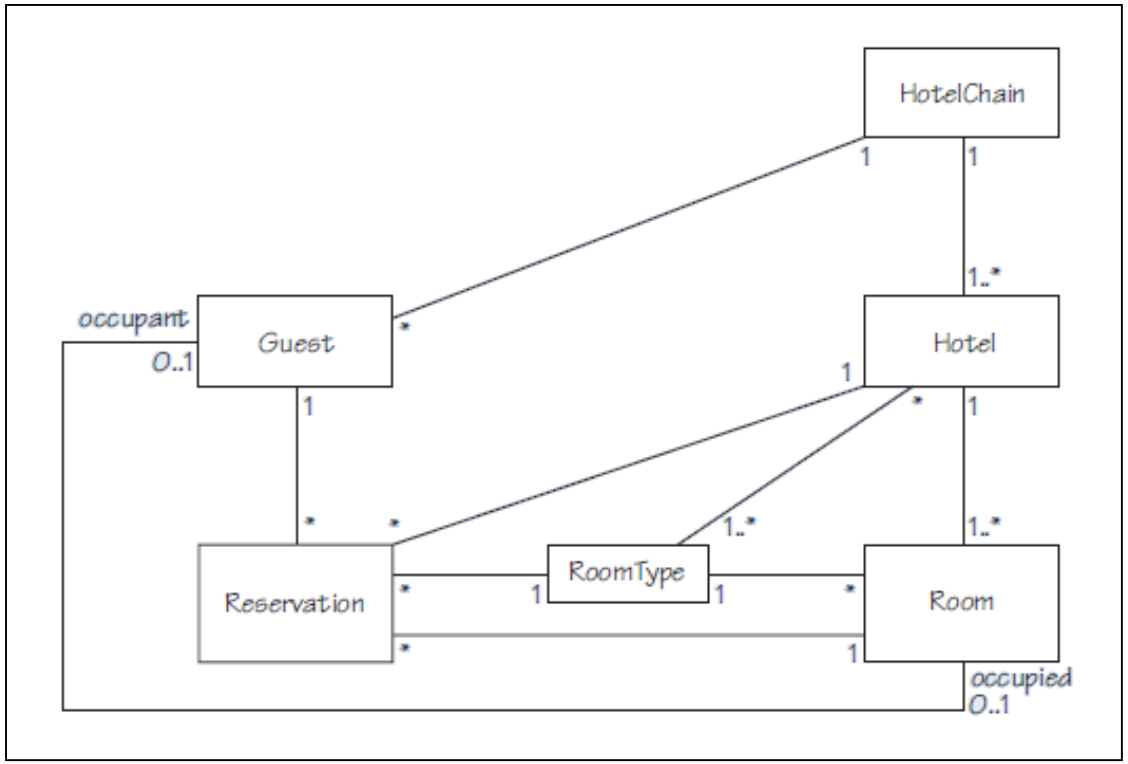


Figure 4 Conceptual model with multiplicities

# Section 2 : Domain modelling – the conceptual model

- A revised conceptual model, with attributes, is shown in Figure 5.
- The attributes types are, at this point, concepts from the problem domain and we do not want to specify how they are represented; examples are `Address`, `CreditCard`, `Money`.
- These attributes may become classes themselves in design and implementation

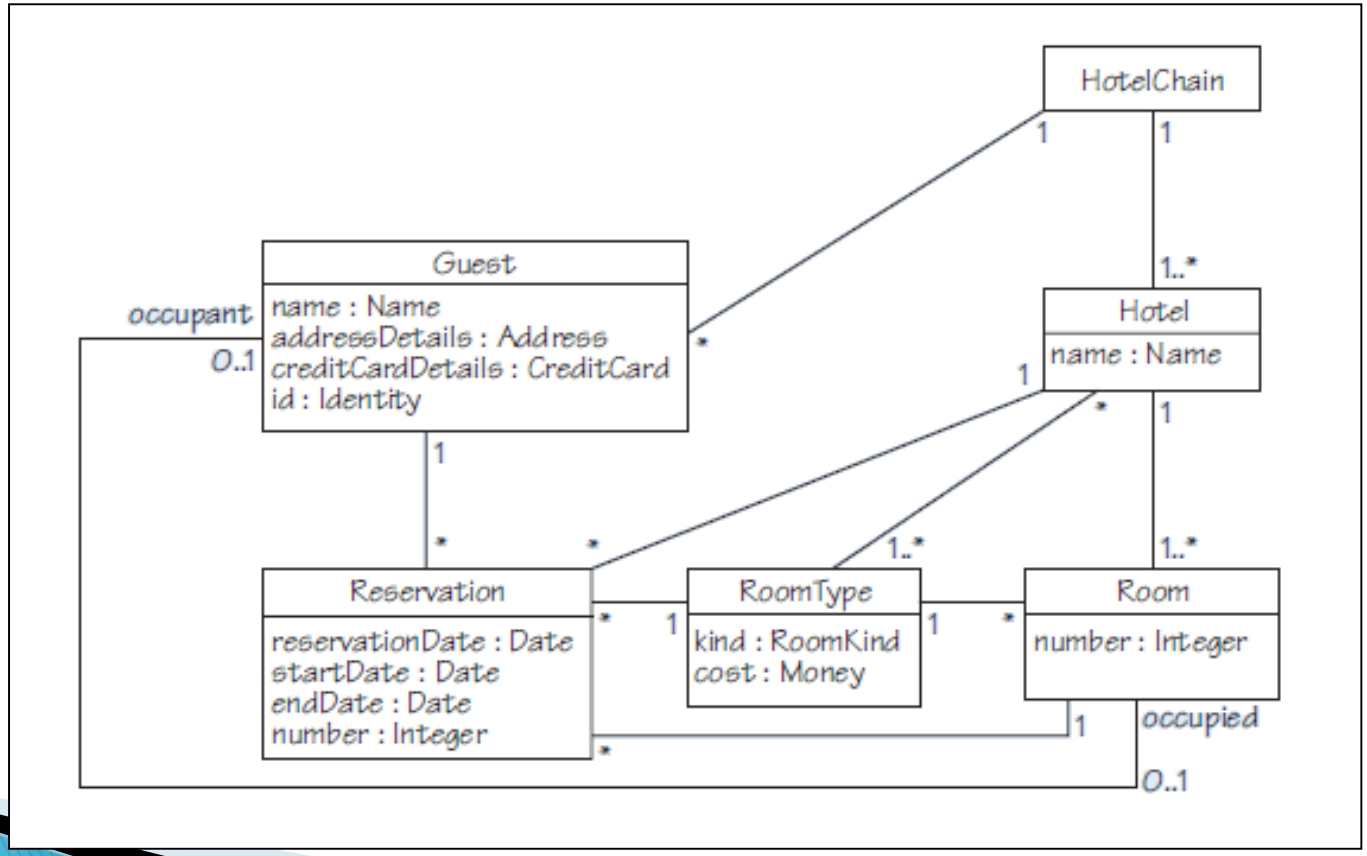


Figure 5 Conceptual model with attributes

# Section 2 : Domain modelling – the conceptual model

Table 9 Adding attributes to the glossary

| Term | Category | Definition |
|---|---|---|
| *name* | Attribute of *Hotel* | The name of a *Hotel*. |
| *name* | Attribute of *Guest* | The full name of a *Guest*. |
| *addressDetails* | Attribute of *Guest* | The address of a *Guest*. |
| *creditCardDetails* | Attribute of *Guest* | The credit card details of a *Guest*, sufficient to make a charge. |
| *id* | Attribute of *Guest* | A unique identifier for a *Guest*. |
| *reservationDate* | Attribute of *Reservation* | The date on which a *Reservation* was made. |
| *startDate* | Attribute of *Reservation* | The date on which a *Reservation* starts. |
| *endDate* | Attribute of *Reservation* | The date on which a *Reservation* ends (the last night). |
| *number* | Attribute of *Reservation* | A unique identifier of a *Reservation*. |
| *number* | Attribute of *Room* | The unique number identifying a *Room* in a *Hotel*. |
| *kind* | Attribute of *RoomType* | The details of a *Room*, in terms of the number of people it can accommodate and the sorts of beds it has (for example *double*, *twin*, *single*, *familyOf5*, *familyOf4*, ...). |
| *cost* | Attribute of *RoomType* | How much the *RoomType* costs per night. This is a simplification, as a *RoomType* may have different costs for different times of the year. |

# Section 2 : Domain modelling – the conceptual model

## Summary of section

- In this section you have seen how the concepts and associations in the conceptual model were selected.
- You have also seen how attributes were identified based on the initial problem statement and the requirements from Block 1 Unit 4.

# Section 3: Analysis

- In analysis we take our conceptual model produced during domain modelling and add various pieces of information.
- First we need to decide whether all concepts correspond to analysis classes and whether there are any classes that need to be represented that were not identified as domain concepts.
- We can also revise our model as we think about its suitability for the task in hand and may need to add association classes.
- The end result will be to produce an analysis model.

# Section 3: Analysis

- We begin analysis by considering constraints on our conceptual model and then switch to a consideration of the system operations.
- During analysis we specify system operations corresponding to use cases.
- We do this by drawing object diagrams relating to the use case steps and then use these diagrams to draw up detailed preconditions and postconditions for system operations – these operations describe the behavior of the system as a black box.

# Section 3: Analysis

## Constraints

- We will identify the constraints on the analysis model by considering association loops in the model.
- Association loop invariants can be expressed from any class context that they touch; so there is a degree of choice in how they are expressed.
- We will also consider any constraints needed on attributes of classes.

# Section 3: Analysis

## Constraint for the loop HotelChain, Hotel, Reservation and Guest

- The Guests associated with the HotelChain, to which the Hotel belongs, should include the Guests for whom the Hotel has Reservations.
- There is no inconsistency following a path from Hotel in one direction (to Guest through HotelChain) and following the path in the opposite direction (to Guest through Reservation).
- In OCL:

```
context Hotel inv:
self.hotelchain.guest-> includesAll (self.reservation.guest)
```

# Section 3: Analysis

## Constraint for the loop Hotel, Room and RoomType

- The collection of Rooms in a Hotel is the same as the collection obtained by pooling the Rooms for all the RoomType objects linked to the Hotel concerned.
- We start again from class Hotel to express this association loop constraint.

- In OCL:

```
context Hotel inv: - - the Rooms of a Hotel are the
Rooms of the RoomTypes of the Hotel
self.room = self.roomType.room
```

# Section 3: Analysis



## Constraint for the loop *Reservation*, *RoomType* and *Room*

The *Rooms* of a *RoomType* for a *Reservation* include the *Room* allocated to a *Reservation*:

```
context Reservation inv:
self.roomType.room->includes(self.room)
```

## Constraints for the loop *Reservation*, *Hotel* and *RoomType*

The *RoomTypes* of the *Hotel* of a *Reservation* include the *RoomType* of that *Reservation*:

```
context Reservation inv:
self.hotel.roomType->includes(self.roomType)
```

*Reservation* also has a constraint on its attributes:

```
context Reservation inv:
(self.endDate > self.startDate) and
(self.startDate >= self.reservationDate)
```

## Constraint for loop *Guest*, *Room* and *Reservation*

If a *Guest* occupies a *Room* in a *Hotel*, the *Room* is allocated to one of the *Reservations* of the *Guest*. However, there might not be any *Rooms* occupied by the *Guest*:

```
context Guest inv:
    self.room->notEmpty()  implies self.reservation.room-
>includes(self.room)
```

# Section 3: Analysis

- Figure 7 shows our analysis class diagram after adding an association class to allow us to record how many Rooms of a given `RoomType`
- The diagram now allows recording guests who neither pay a bill nor make a reservation. For example, partners and children.

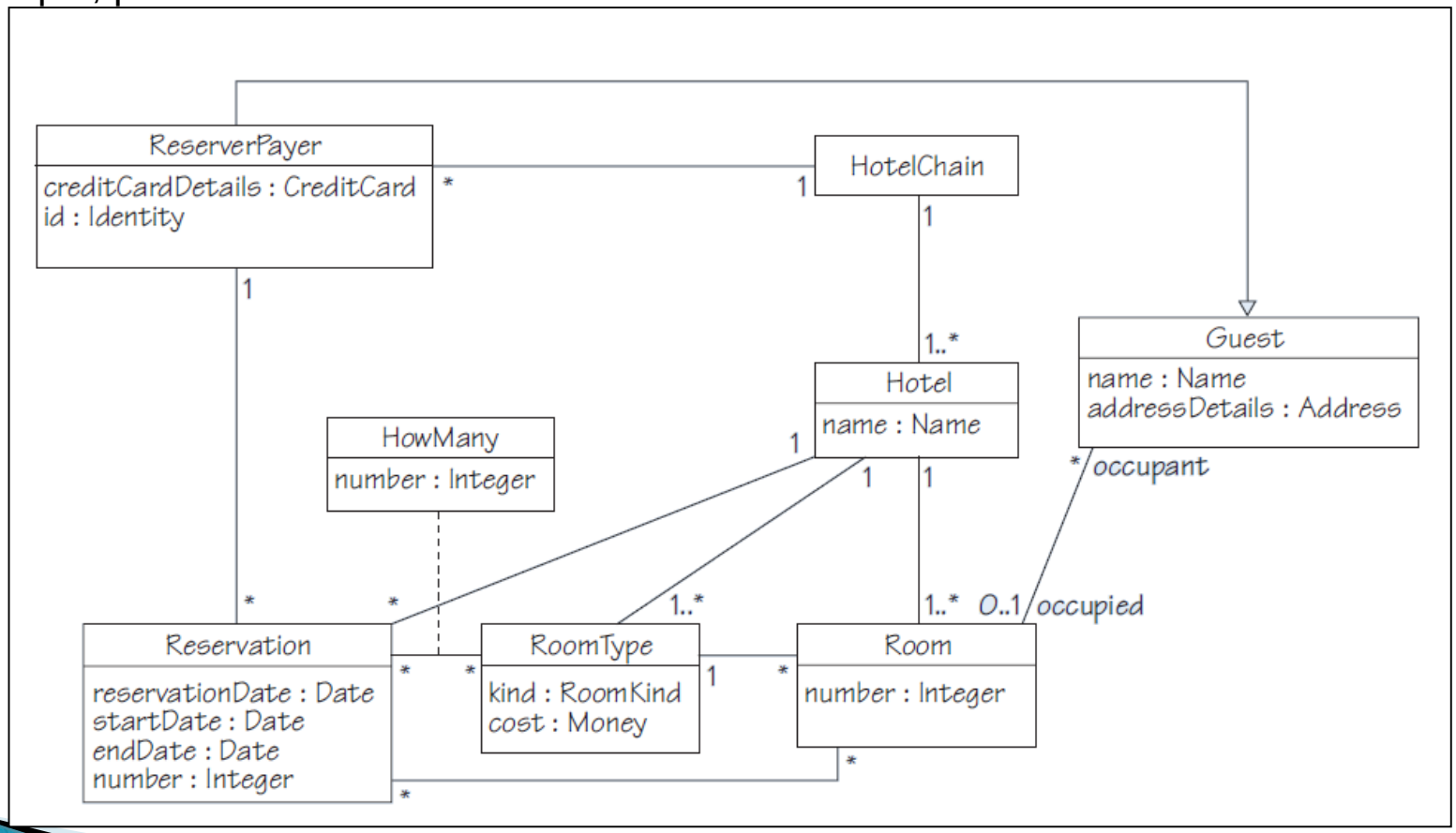


Figure 7 Analysis class diagram

# Section 3: Analysis

- Updating glossary

Table 10 Updating concepts in the glossary

| Term | Category | Definition |
|---|---|---|
| *Guest* | Concept | Somebody who might stay in a *Hotel*. |
| *ReserverPayer* | Concept | A *Guest* who reserves and/or pays for a *Room*. |
| *HowMany* | Concept | An association class to record the number of *Rooms* of a given *RoomType* in a *Reservation*. |

Table 11 Updating associations in the glossary

| Term | Category | Definition |
|---|---|---|
| *Guest–Room* | Association | A *Guest* can occupy a *Room*. |
| *ReserverPayer–HotelChain* | Association | A *ReserverPayer* can be a customer of the *HotelChain* (has a past, current or future reservation for some hotel of the chain). |
| *ReserverPayer–Reservation* | Association | A *ReserverPayer* can have a number of *Reservations*. These include current *Reservations* (those covering a future stay, but also including the period when the *ReserverPayer* is staying in the Hotel) and past *Reservations*, which are recorded for reasons outside the current domain analysis. |

Table 12 Updating attributes in the glossary

| Term | Category | Definition |
|---|---|---|
| *number* | Attribute of *HowMany* | The number of *Rooms* of a given *RoomType* in a *Reservation*. |
| *name* | Attribute of *Guest* | The full name of a *Guest*. |
| *addressDetails* | Attribute of *Guest* | The address of a *Guest*. |
| *creditCardDetails* | Attribute of *ReserverPayer* | The credit card details of a *ReserverPayer*, sufficient to make a charge. |
| *id* | Attribute of *ReserverPayer* | The identity of a *ReserverPayer*. |

# Section 3: Analysis

## System operations

- System operations describe the behavior of the system as a black box.
- These operations define the external **interface** of the system (i.e. what any implementation will have to provide to the user, independently of how it is provided).
- The specifications of operations can be written in English, adopting the conventions that a precondition is described in the present tense and a postcondition is described in the future tense.

# Section 3: Analysis

- The syntax for an operation specification is as follows:

```
context TypeName::operationName(parameter1 :
Type1,…) : ReturnType
pre:
- - condition
post:
- - condition
```

  - An operation is defined within a context (TypeName is the class in which the operation is defined) and with a name (operationName), a series of parameters and their types, and the type of the returned value (when there is a returned value); this is the operation's signature.

## Specifying operations

- At this point in the design it is convenient to allocate all the identified system operations to the **system class**.
- **A system class** is just a convenient device to help us make progress with the system design.
- The system class in our example is `HotelChain`, which:
  - represents the whole system;
  - has a single instance;
  - will coordinate the behavior of other objects in the system in order to carry out the operations.

# Section 3: Analysis



- In large systems the responsibility of the system class is typically distributed among a number of separate classes.
- The system class has one defining characteristic: there is only one instance of such a class.
- The system class keeps track of objects in the system via associations either directly or indirectly.
  - Directly through direct association as in the case for the class Hotel.
  - Indirectly through the associations that the Hotel class has with other classes in the system.

# Section 3: Analysis

Table 13 shows the system operations of the make reservation use case.

Table 13 The *makeReservation* system operation

| | |
|---|---|
| **Identifier and name** | UC1 *make reservation* |
| **Initiator** | *Reserver* or *Receptionist* |
| **Goal** | A room in a hotel is reserved for a guest. |
| **Precondition** | None (that is, there are no conditions to be satisfied before carrying out this use case). |
| **Postcondition** | A room of the desired type will have been reserved for the guest for the requested period, and the room will no longer be free for that period. |
| **Assumptions** | The expected initiator is a reserver or receptionist, using a web browser to perform the use case. The guest is not already known to the hotel's software system (see step 5). The receptionist (if an actor) communicates details to the client when making a booking. |
| **Main success scenario** | |
| 1 | The reserver/receptionist makes a reservation request. |
| 2 | The reserver/receptionist selects the desired hotel, dates and type of room. |
| 3 | The hotel system provides the availability and price for the request; that is, one or more offers are made. |
| 4 | The reserver/receptionist accepts the offer. |
| 5 | The hotel system requests identification and contact details. |
| 6 | The reserver/receptionist provides identification and contact details of the guest for the hotel system's records, as well as credit card details. |
| 7 | The hotel system creates a reservation and gives it a number. |
| 8 | The hotel system allocates a room to the reservation. |
| 9 | The hotel system reveals the reservation number to the reserver/ receptionist. |
| **Extensions** | |
| 3.a.1 | *No availability in this hotel.* The hotel system offers alternatives at a different hotel or hotels within a limited radius. |
| 3.b.1 | *No availability in this hotel or in any other hotels within a limited radius.* The hotel system informs the reserver and terminates the use case. |
| 4.a.1 | *Offer not accepted.* The hotel system terminates the use case. |
| 6.a.1 | *Guest already on record.* The guest is identified and (if needed) their record is updated and the scenario continues at step 7. |

# Section 3: Analysis

Example: Consider the snapshot in Figure 8 and draw a snapshot after the execution of the system operation `makeReservation(hotel1, "8/4/2013", "10/4/2013", roomType2, reserverPayer3)`. Assume that the allocated room is `room1`.

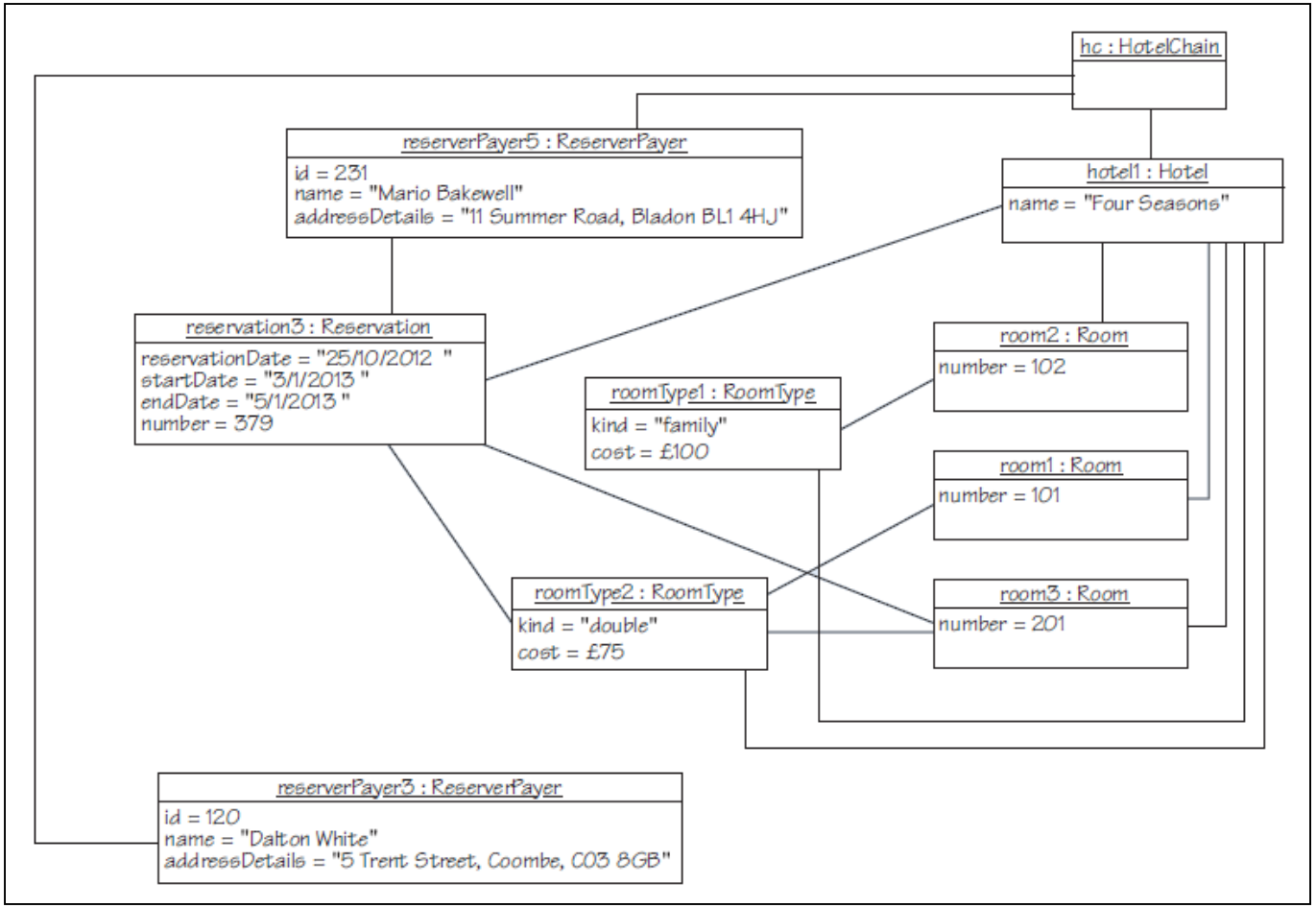


Figure 8 Snapshot illustrating a sample state of the system before the makeReservation operation is executed

# Section 3: Analysis

**Answer**

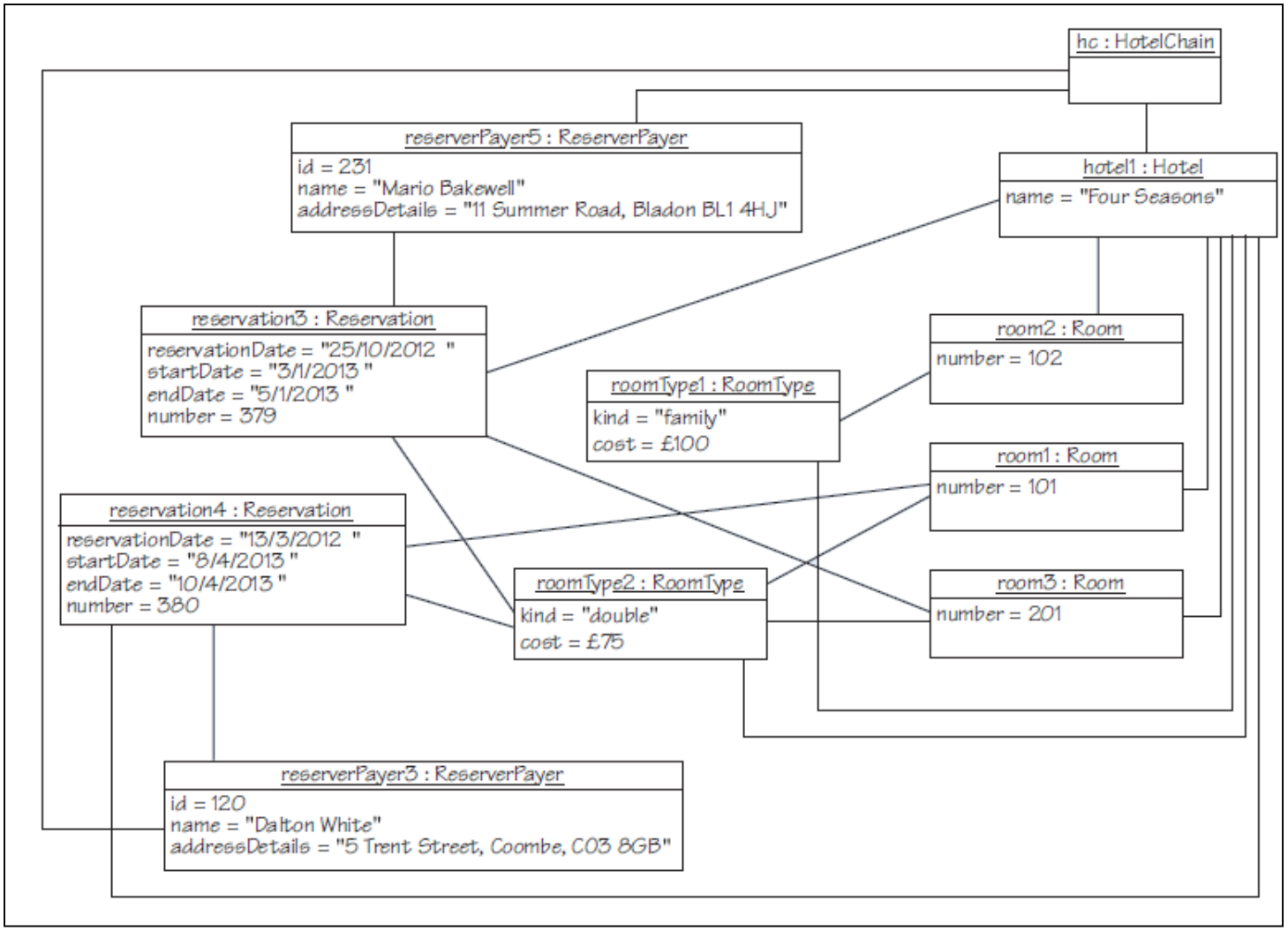


Figure 9 Snapshot illustrating the state after another reservation has been made: makeReservation(hotel1, "8/4/2013", "10/4/2013", roomType2, reserverPayer3)

# Section 3: Analysis

## Complete specification of the makeReservation operation

**context** HotelChain::makeReservation(hotelName : Name, startDate : Date, endDate : Date, roomType : RoomType, reserverPayer : ReserverPayer) : String

**pre:**

- - startDate is before endDate
- - startDate is not in the past
- - roomType is one of hotel's roomTypes

**post:**

- - if available(startDate, endDate, roomType) executed in hotel returned true then
- - a new Reservation object, reservation, will have been created (object creation)
- - hotelChain will be linked to reserverPayer
- - hotel will be linked to reservation
- - reserverPayer will be linked to reservation
- - reservation will be linked to roomType
- - reservation will be linked to a previously free Room
- - a string indicating the reservation number will have been returned
- - otherwise, a string indicating failure will have been returned

- *Specifications of cancelReservation system operation and checkInGuest system operation can be found in the course material*

## Summary of section

- In this section you have seen how to place constraints on the structural model.
- You have also seen how to relate the specifications of system operations to the use cases, and the effects on the structural model of the system..

# Section 4: Design – assigning responsibilities

- In design, the analysis model defined during requirements analysis is refined in a number of ways to become the design model.
- More responsibilities are assigned to classes, by applying basic design principles and by analyzing the pre- and post-conditions of system operations.
- The navigation of associations is established, based on the way objects in the system need to collaborate to realize the system operations.
- Operations are added to the classes, based on the assigned responsibilities.

# Section 4: Design – assigning responsibilities

- For the remainder of this unit, we will use the class diagram for the HotelChain introduced in subsection 3.1 as Figure 7 and reproduced here, with the addition of the HotelChain operations, as Figure 13

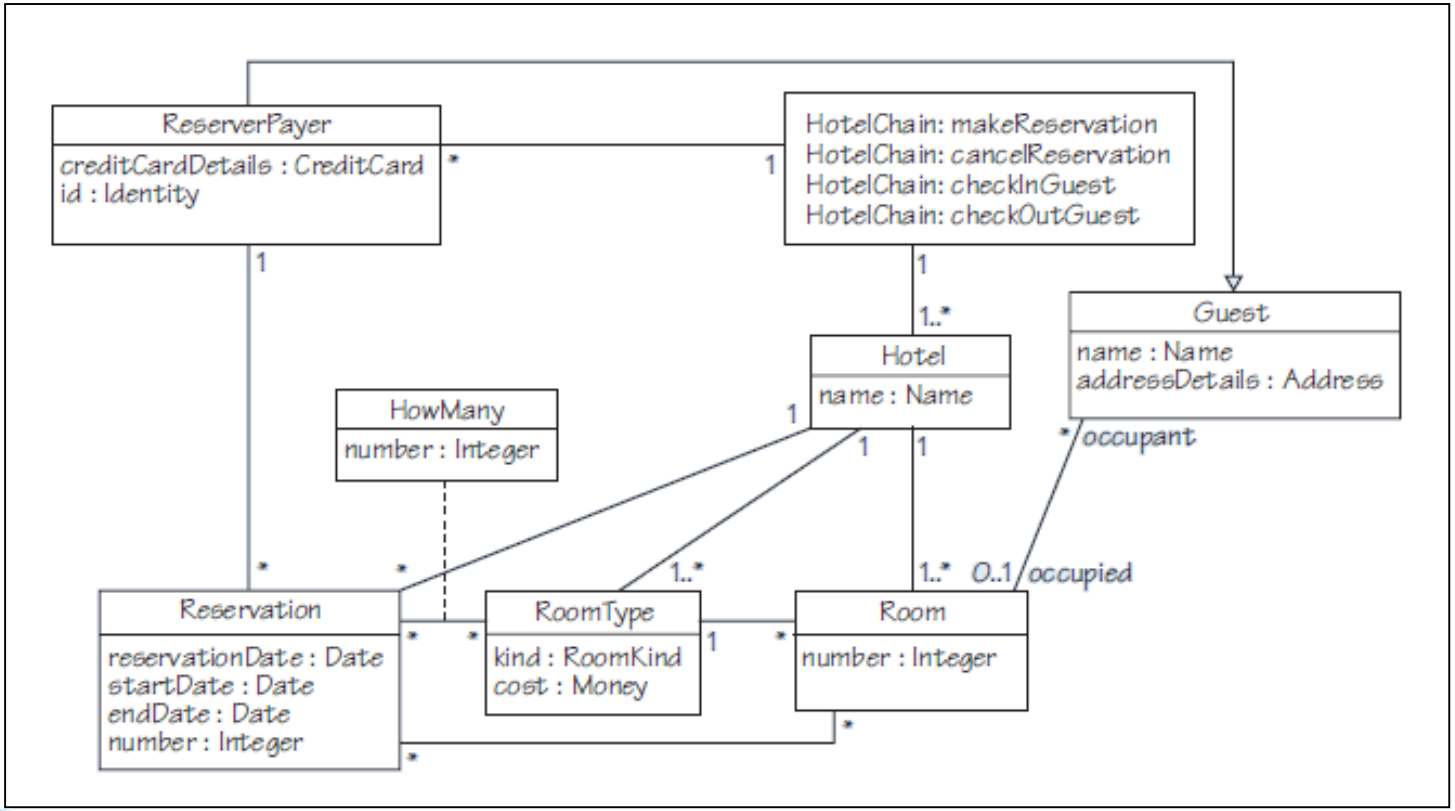


Figure 13 HotelChain class diagram

# Section 4: Design – assigning responsibilities

## Value identities

- We need a more meaningful way to distinguish between objects, one that can be used in software.
- Many objects can be distinguished from each other by the values of one or more of their attributes. This form of identity is known as **value identity.**
- Value identities on a class diagram can be represented by using the UML notation of **qualified association.**
- **I**n a qualified association, a qualifier explicitly indicates the set of object properties that represents the identity of objects at the opposite end of the association.

# Section 4: Design – assigning responsibilities

In Figure 14 the class diagram has been redrawn to show value identities for the various classes: hotels are distinguished by `name`, rooms in a hotel by `number`, reservations by `number`, and `ReserverPayers` by `id`.

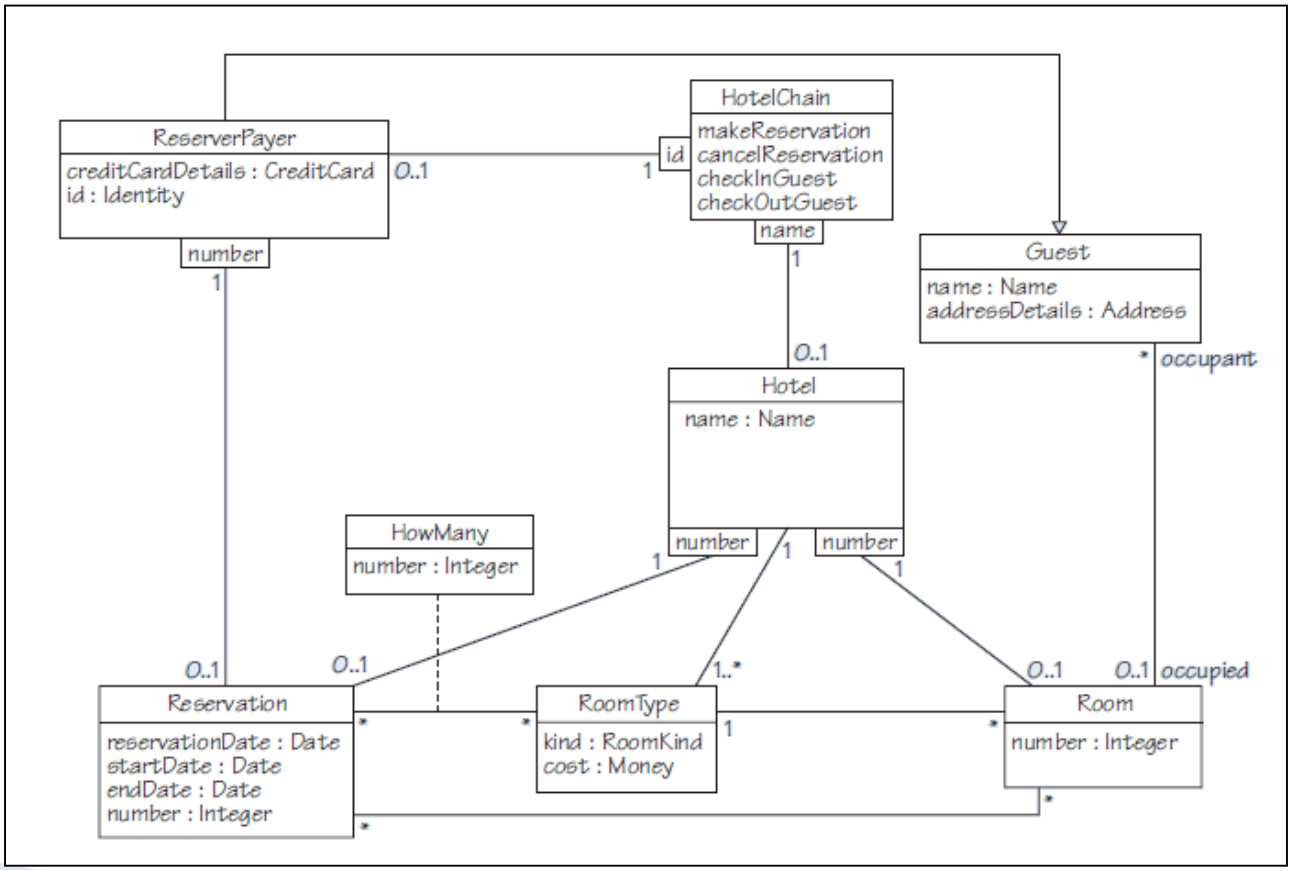


Figure 14 HotelChain class diagram with qualified associations

# Section 4: Design – assigning responsibilities

## Verifying and fulfilling preconditions and postconditions

- By using pre- and postconditions, we can rigorously describe the conditions under which the service provided by an operation can be used, and the result obtained after the service is provided.
- Writing pre- and postconditions in a rigorous way also has the advantage of making us think about the requirements, clarifying any ambiguity and rectifying any omissions.
- A rigorous postcondition is also the first step for verification and testing.
- A properly implemented system will successfully make the postcondition of an operation true whenever that operation is executed, provided that the matching precondition is satisfied.

# Section 4: Design – assigning responsibilities

## Verifying preconditions

- Preconditions are verified, and postconditions fulfilled, by objects interacting with each other in a predefined way that may result in objects, and links between those objects, being created or destroyed

- Verifying preconditions may require consideration of:
  - whether particular objects exist
  - whether particular links exist
  - the values of particular attributes

- Fulfilling postconditions may involve:
  - creating particular objects
  - creating particular links
  - modifying the values of particular attributes
  - returning particular values

# Section 4: Design – assigning responsibilities

## Example: Consider the makeReservation operation

- We start by focusing on the precondition of this operation. In order to identify the parties involved in the collaboration to verify the precondition, we need to identify:
  - who initiates the collaboration (this is the actor of the corresponding use case; in the case of make reservation it is a Reserver)
  - the object that orchestrates the collaboration (this is by definition the system object)
  - other objects in the system with which the system object needs to collaborate in order to verify that the precondition of the operation is satisfied (to be able to identify these objects we need to look at the operation's definition).

# Section 4: Design – assigning responsibilities

- By looking at the arguments and the precondition of the operation, we can see that the objects involved are the hotels, rooms and RoomType objects in the system.
- The hotels are linked to the system object through a qualified association, capturing the fact that the system object can perform a mapping between the value identity – the name of the hotel in this case – and the identity of the corresponding object.
- The details of this mapping will be addressed at implementation, when features of the target programming language will be used to realise the mapping.
- The mapping need not be considered any further in design: all the information needed is already captured in the qualified association.
- Similarly, Room objects are linked to Hotel objects by qualified associations with the qualifier number, and so on.

# Section 4: Design – assigning responsibilities

- We will now access objects by their value identities where qualified associations have been added to the class diagram in Figure 14.

```
context HotelChain::canMakeReservation (hotelName : Name, startDate : Date, endDate : Date, roomType : RoomType) : Boolean
post:
- - true will be returned if
- - startDate is before endDate
- - startDate is not in the past
- - there is a hotel with Name hotelName, and it has rooms of roomType)
- - otherwise, false will be returned
```

Note: Operations for verifying the preconditions of `cancelReservation` and `checkInGuest` can be found in the course material

# Section 4: Design – assigning responsibilities

## Fulfilling postconditions

- We can now consider the collaborations to satisfy the postcondition of a system operation.

- We will consider the makeReservation operation from HotelChain in order to illustrate the technique. The objects that are involved in the collaboration, and their links at makeReservation's postcondition

  - *The system object, a Hotel object, a ReserverPayer object (either pre-existing or newly created), a RoomType object, a Room object and the newly created Reservation object.*

# Section 4: Design – assigning responsibilities

## Fulfilling postconditions

- Fulfilling the postcondition of the operation makeReservation corresponds to the creation of objects and the creation of new links between objects.
- To show this, we choose to use communication diagrams rather than sequence diagrams, because the communication diagrams clearly show the creation of new objects and new links with the {new} constraint.

# Section 4: Design – assigning responsibilities

- The responsibility for the fulfilment of makeReservation will ultimately lie with Hotel.

- The operation signature of the operation of the class Hotel that is relevant to fulfil the postcondition of makeReservation as follows:

```
context Hotel::available(startDate : Date,
endDate : Date, roomType : RoomType) : Boolean
```

- By applying GRASP Creator, the creation of a new ReserverPayer should be assigned to HotelChain, as HotelChain records and holds information for all guests that ever come into contact with the hotel chain.

# Section 4: Design – assigning responsibilities

- As for a new reservation, the responsibility may be assigned to either Hotel or ReserverPayer: either of these classes may record information about a reservation

```
context HotelChain ::createReserverPayer (Name, address : Address, creditCardDetails : CreditCard) : ReserverPayer
```

```
context Hotel::createReservation(startDate : Date, endDate : Date, roomType : RoomType, reserverPayer : ReserverPayer) : Reservation
```

# Section 4: Design – assigning responsibilities

- An object is created by a class operation, usually known as the constructor operation.
- The constructor operation returns the newly created instance, that is, an object identity for the newly created object.
- The operations `createReserverPayer` and `createReservation` will invoke the create class operations of `ReserverPayer` and `Reservation` respectively.

```
context ReserverPayer::create(name : Name, address : Address, creditCardDetails : CreditCard) : ReserverPayer
```

# Section 4: Design – assigning responsibilities

The communication diagram of Figure 15 shows the interactions that fulfil the postcondition of `makeReservation` when `reserverPayer` is not known to the hotel, and when there is availability of the requested `roomType`.

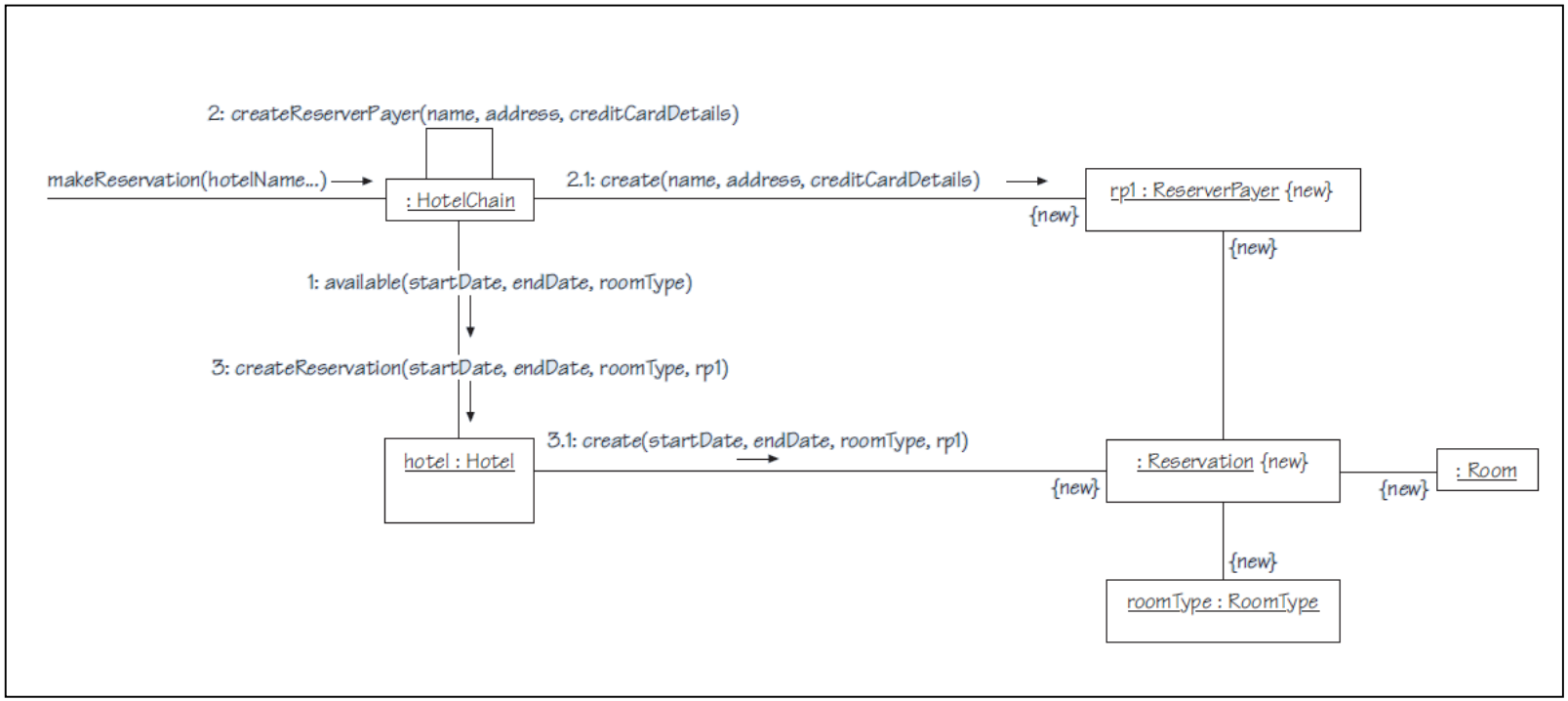


Figure 15 Communication diagram for the fulfilment of the postcondition of makeReservation

# Section 4: Design – assigning responsibilities

- The communication diagram of Figure 15 shows the interactions that fulfil the postcondition of `makeReservation` assuming that the responsibility for creating a reservation is assigned to `ReserverPayer`.

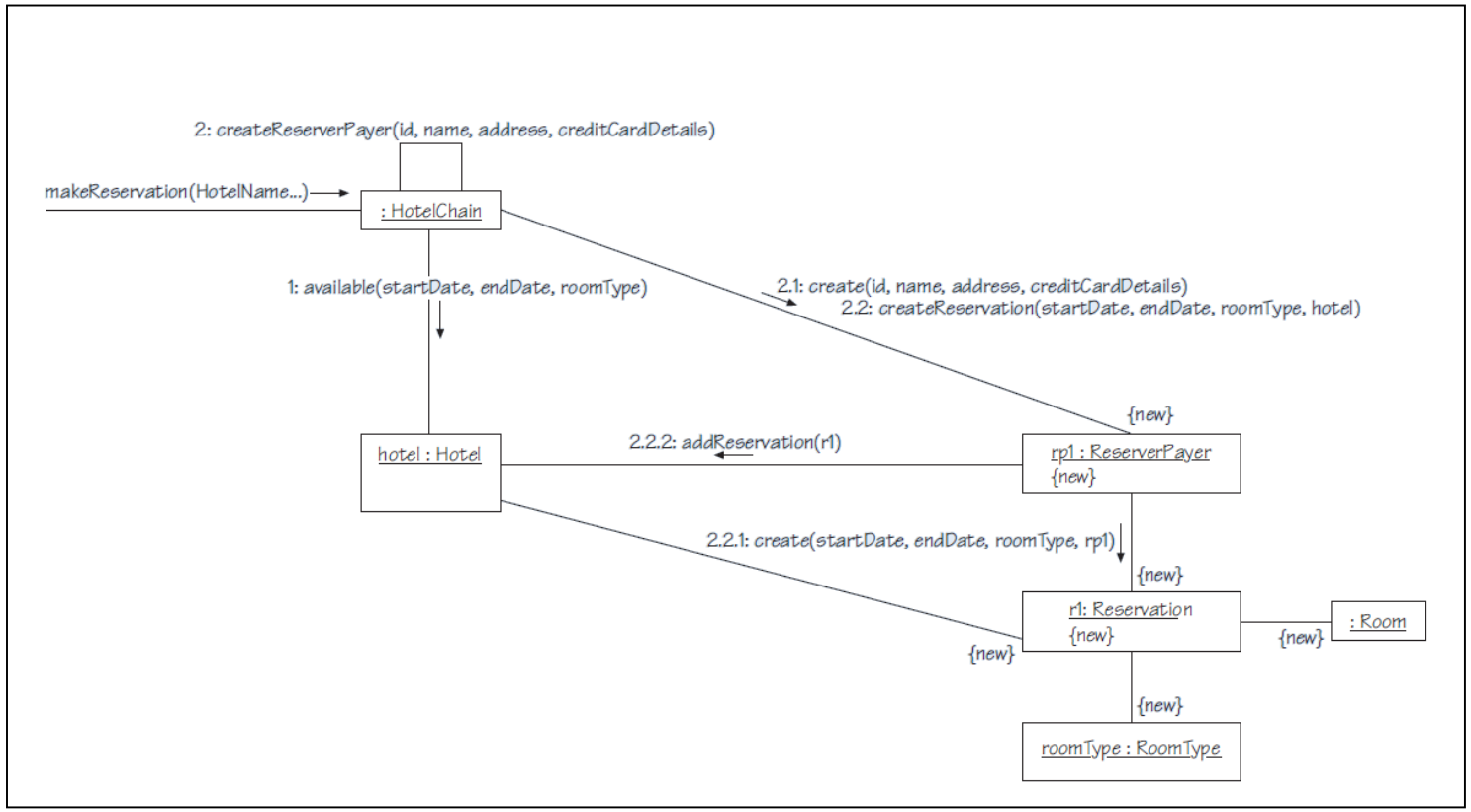


Figure 16 Revised communication diagram for the fulfilment of the postcondition of makeReservation

# Section 4: Design – assigning responsibilities

- The communication diagram of Figure 15 shows the interactions that fulfil the postcondition of `makeReservation` assuming that the `ReserverPayer` is known to the `Hotel`.

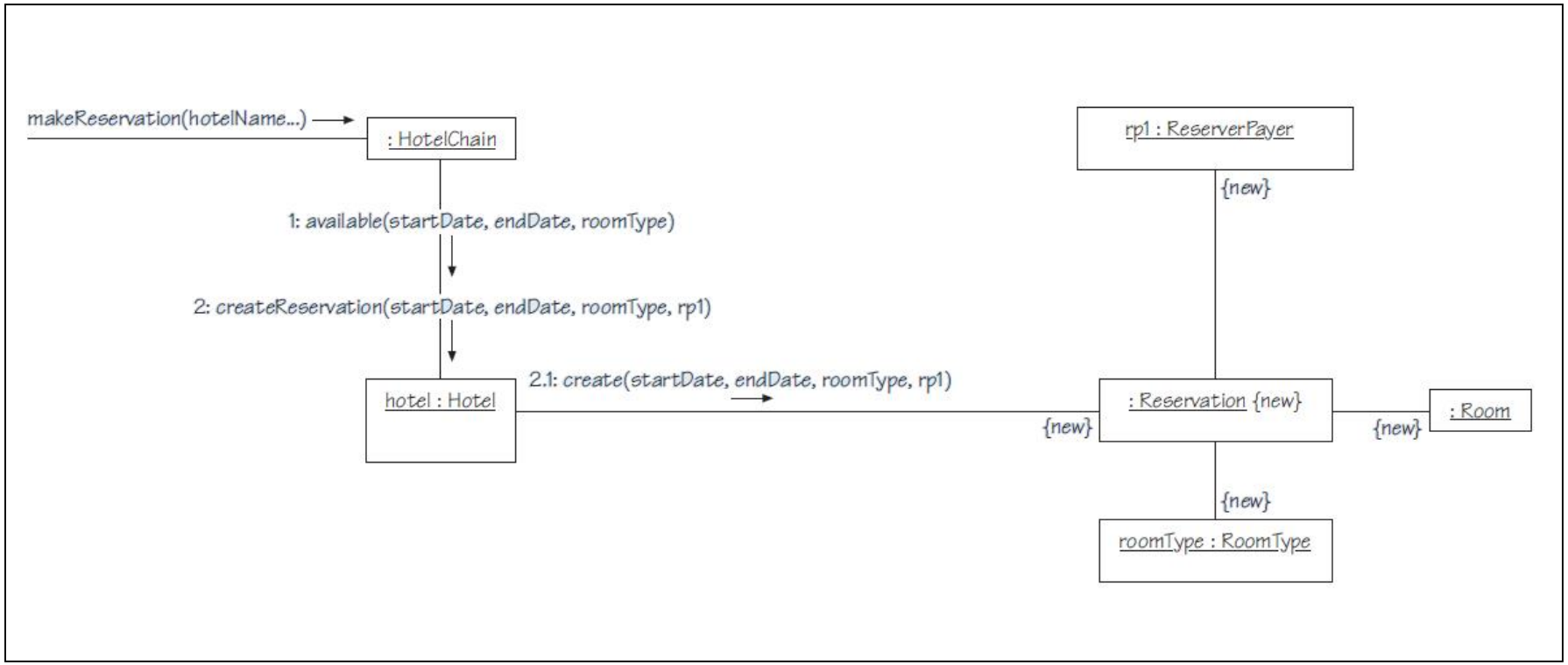


Figure 17 Revised communication diagram for makeReservation, assuming that the ReserverPayer is known to the Hotel

# Section 4: Design – assigning responsibilities

## Structural design model

- We can now redraw the class diagram to take into account the allocation of operations to classes that we have carried out so far, along with some tentative decisions in relation to the navigation of operations
- We need to add the operations below:

*HotelChain::makeReservation*

*HotelChain::cancelReservation*

*HotelChain::checkInGuest*

*HotelChain::checkOutGuest*

*HoteChain::createReserverPayer*

*HotelChain::canMakeReservation*

*HotelChain::canCancelReservation*

*HotelChain::canCheckInGuest*

*HotelChain::canCheckOutGuest*

*Hotel::createReservation*

*Hotel::available*

*Room::createGuest*

*ReserverPayer::create*

*Reservation::create*

*Guest::create*

# Section 4: Design – assigning responsibilities

- Figure 18, indicated the visibility of operations.
- The UML notation of a + to indicate public methods and a – to indicate private methods.
- We need the system operations to be available to classes outside the package containing them, but where an operation is called from the same class it can be private.

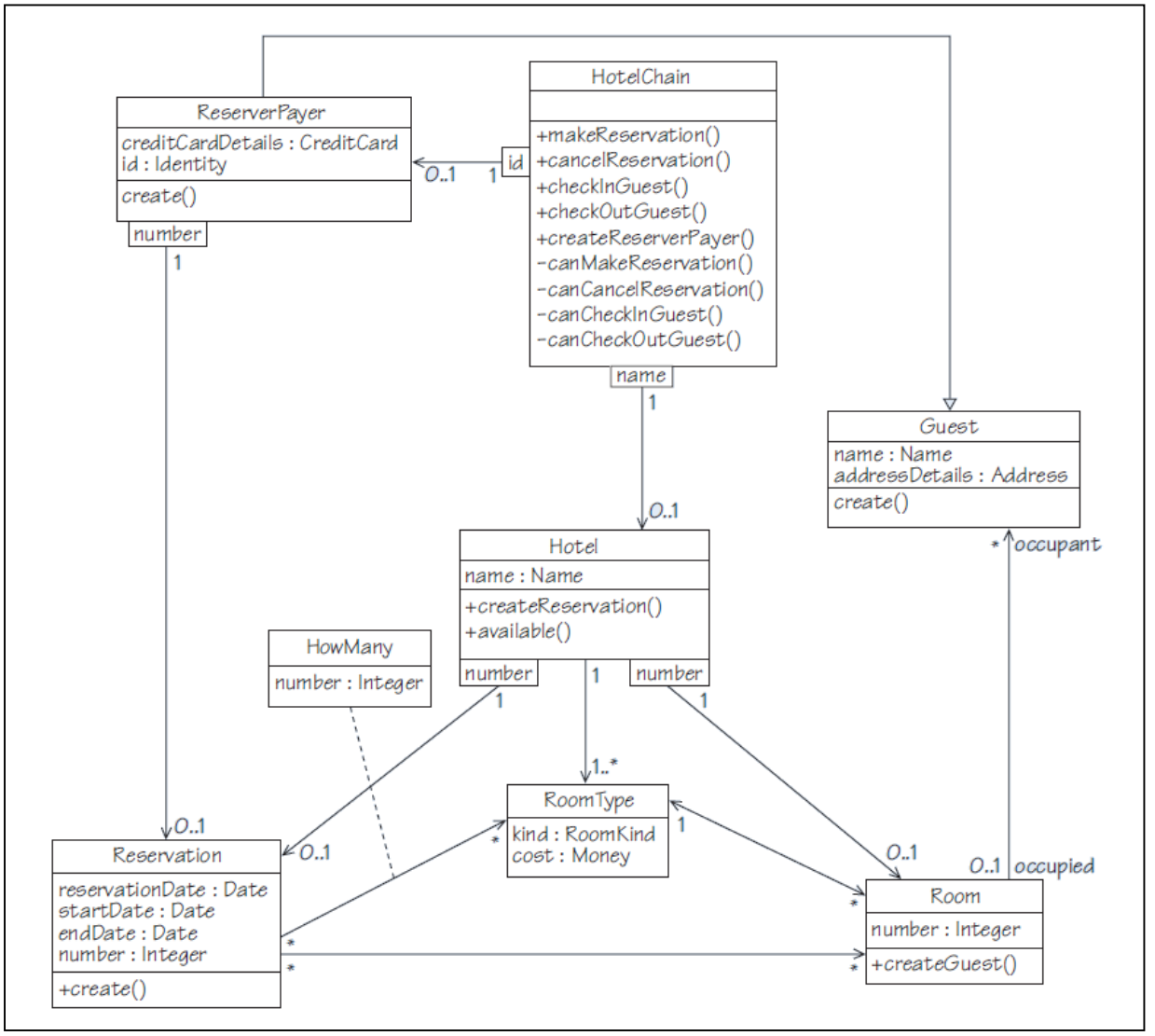


Figure 18 Structural design model with operations shown

# Section 4: Design – assigning responsibilities

## Exercise

- What if we have a client who has worked in a hotel where the computer system managed to lose track of rooms? If they asked us to consider this problem we could think about using a state machine to model a typical room object.
- Draw a state machine diagram to show the states of a room and how the room moves between them.

## Solution

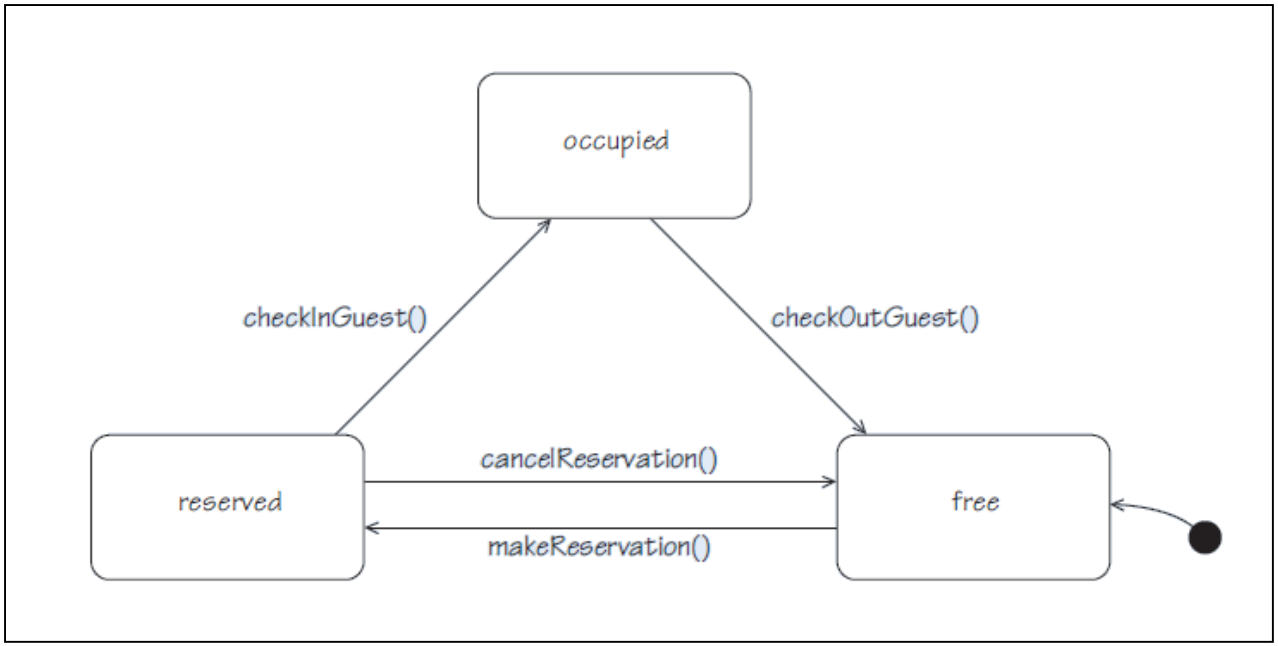


Figure 19 State chart for a Room object

# Section 4: Design – assigning responsibilities

**Summary of section**

- In this section you have seen how system operations were allocated to classes, and how their pre- and postconditions are fulfilled.
- You have also seen how navigability was added to the class diagram.

# Section 5: Consistency

- In this case study we have carried out some analysis and design, to arrive at specifications for a set of operations that represent the functionality demanded by our requirements.

- These operations have now been assigned to appropriate classes, and some of the navigability of links that will be required has been tentatively established.

- Once we arrived at an analysis class model, we carried out three main steps to decide how the requirements from the use cases could be supported by this structure.
  1. We specified one system operation per use case (with pre- and postconditions).
  2. Using interaction diagrams where appropriate, we designed operations to verify preconditions and fulfil postconditions of the system operations – leading to identification of finer-grained operations.
  3. We assigned the operations to classes by revisiting the class diagram and also established navigability of links. This has led us to a design model.

# Section 5: Consistency

## Need of Consistency check

- We should think about the consistency in the context of the case study.
- Given that we have followed a fairly systematic process to arrive at our design, you might believe that consistency is guaranteed.
- However, it is easy to make mistakes, and in a large project and under time pressure it might well be the case that operations, and indeed classes and links, get added in a more ad hoc way to ‘get the job done’.

# Section 5: Consistency

**Some consistency checks that we might apply are the following.**

1. Do the system requirements in Block 1 Unit 4 all correspond to use case steps, and conversely is there at least one system requirement for each step?
2. Have we done enough at this stage to realise the use cases in our class diagram? In other words do the use cases link to the class diagram fully?
   a. Is there one system operation for each use case?
   b. Do the operations in the design class model correspond to operations identified in design either to verify preconditions or to assert postconditions of system operations? Where we built explicit dynamic models (interaction diagrams), the operations in the models should correspond to operations on the class diagram.
   c. Do the operations identified create only links that correspond to associations in the class diagram?
   d. Do the operations identified modify only attributes of the classes where they are defined, or that were passed as arguments to the operation?
3. We have used state machines to understand the behaviour of one of our classes. Any events in a state machine have to correspond to operations of the corresponding class.

# Section 5: Consistency

- One approach to consistency is the use of walkthroughs.
- For example, we can read through each step of a use case scenario to check that our design matches. This involves:
  - going through the steps of the use case scenarios and the relevant operations and any corresponding dynamic models (interaction diagrams)
  - following the paths of operations and associations in the design class diagram.
- In this way we can check whether our design supports our use cases and revisit any decisions that have compromised consistency.

# Section 5: Consistency

- There is a choice about whether we do these two steps together or separately.
- Early on in a development it might be helpful to work in the context of the use cases.
- Later on we might apply the checks independently, to reduce cost.
- If we find errors, we will still need to be sure we have dealt with them right across the development process.

# Section 5: Consistency

## Exercise

What would lead you to conclude that an interaction diagram and a class diagram were inconsistent?

## Solution

There would be an inconsistency if we found a message in an interaction diagram being sent to a class and:

1) there was no corresponding class in the class diagram
2) or it didn't support the message
3) or the message could not be sent because of navigability issues.

# Section 5: Consistency

## Exercise

Having discovered an inconsistency, what would you do next?

## Solution

- Given an inconsistency, we need to consider the source of the error. It could be that we have simply failed to allocate an operation in the class diagram or placed it in the wrong class. This would be easy to fix, but we need to make sure that such a fix does not lead to inconsistencies with other dynamic models.
- There could be a rational reason why the class diagram is the way it is and the dynamic modelling is wrong. As well as correcting the dynamic model we should also check that it remains consistent with, for example, the use cases.
- If fixing one inconsistency leads to others we might like to consider whether our overall approach is sensible.

# Unit Summary

- This unit has followed an iteration of domain modelling, through requirements and analysis to design.
- In Block 1 Unit 4 we looked into the requirements for this case study and decided on a subset for a first iteration.
- In this unit we proceeded by building a structural model of the domain, evolving it to a structural analysis model by taking decisions on what needs to be represented in a software solution and considering constraints.
- Finally we identified the system operations for the requirements chosen and carried out the allocation of responsibilities to classes taking design decisions.

# Unit Summary

On completion of this unit you should be able to:

- carry out domain modelling by identifying concepts, associations, multiplicities and attributes
- carry out analysis by taking decisions on what needs to be represented in a software solution and specifying constraints and identifying system operations
- carry out design by identifying further system operations, establishing pre and post conditions, and assigning system operations to classes by taking design decisions.